

Representacion de datos

Los datos recursivos se representan de dos formas

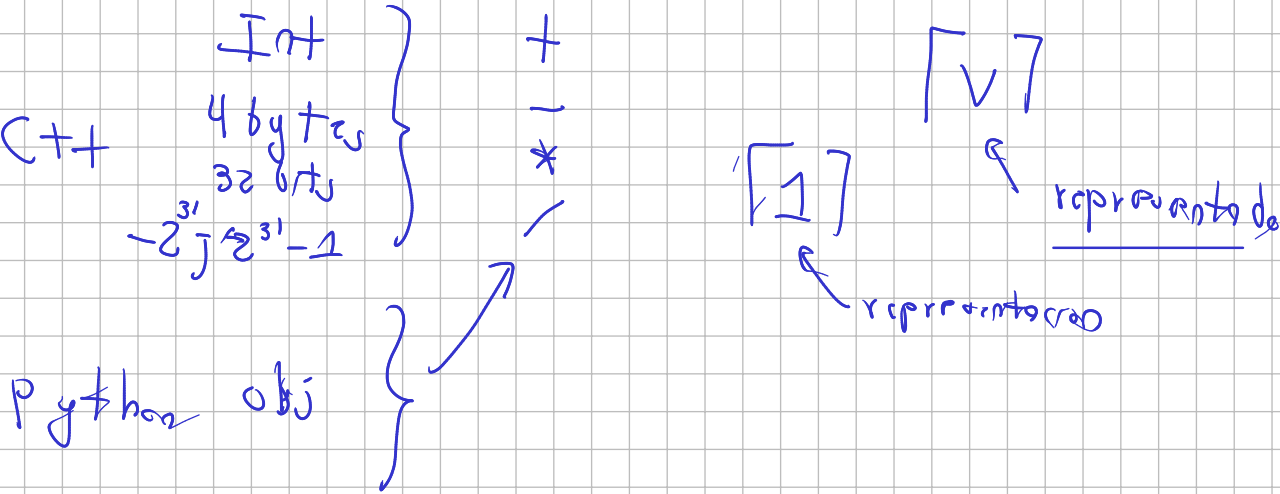
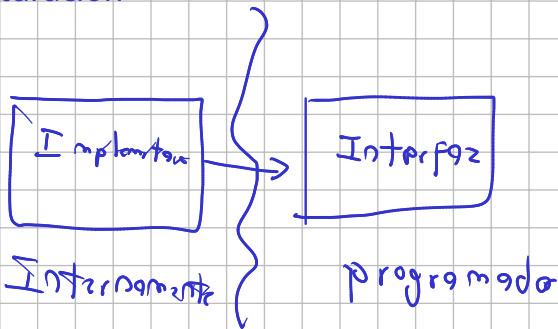
1) Inductiva

$$\begin{matrix} c \in S & n \in \mathbb{N} \\ \hline l \in S \wedge n \in \mathbb{N} \\ n.l \in S \end{matrix}$$

2) Gramaticas

$$\langle l s t \rangle ::= 'c' \mid \langle int \rangle \langle l s t \rangle$$

Estructuración



Construccion de TADS dado una gramtica

Procedimientos observadores

Que permiten interactuar con el TAD

- Constructores: Construye una instancia del dato
- Predicados: Saber si un dato pertenece al TAD

Procedimientos extractores

Extraen información dentro del TAD

```
<enviroment> ::= '()  
    empty-env()  
    :: (lista id) (lista valores) <environment>  
    extend-env(lid lval old-env)
```

Constructores

1. empty-env construye un ambiente vacio
2. extend-env construye un ambiente extendido, recibe tres parametros lid, lval, old-env

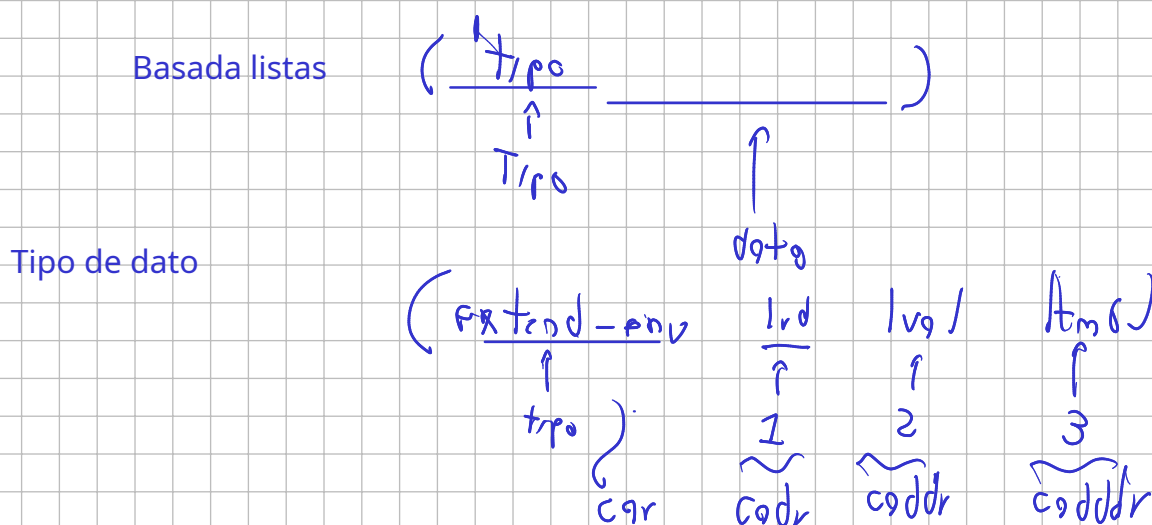
Predicados

1. empty-env? Validar si es un ambiente vacio
2. extend-env? validar si es un ambiente extendio

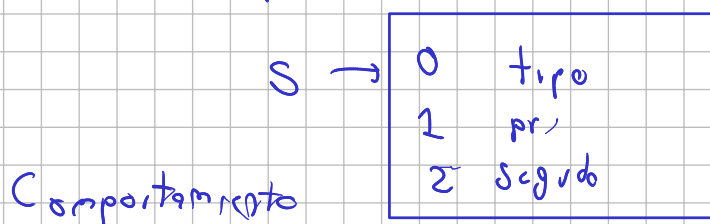
Extractores

1. extend-env->lid
2. extend-env->lval
3. extend-env->old-env

Este unicamente tiene un procedimiento del area del programado llamado apply-env



Banda procedimientos



```

<lc-exp> ::= <identifier>
           var-exp(id)
           ::= "lambda" "(" <identifier> ")" <lc-exp>
           lambda-exp(id exp)
           ::= <lc-exp> <lc-exp>
           app-exp(rator rand)

```

1. ¿Que procedimientos me ofrece este TAD?

1.1 Constructores

var-exp lambda-exp app-exp

1.2 Observadores

1.2.1 Predicados var-exp? lambda-exp? app-exp?

1.2.3 Extractores var-exp->id, lambda-exp->id, lambda-exp->exp, app-exp->rator, app-exp->rand

Area programador

1. occurs-free?

2. occurs-bound?

```

(define e
  (lambda-exp 'e
    (app-exp
      (var-exp 'x)
      (lambda-exp 'p
        (var-exp 'y))))))

```

